Homework 1 Report

Yutong Wang
April 17, 2026

---

1 Setup and Execution

1.1 Dependencies and Installation

List all required packages and how to install them.

```
pip install langgraph langchain openai pandas
```

Listing 1: Installation

1.2 Configuration

Create a `.env` file in the project root directory with OpenAI API key

```
OPENAI_API_KEY = sk-**********
MODEL_NAME = gpt-5.4-mini
```

Listing 2: .env file

The agent loads the key automatically at runtime via `python-dotenv`.

1.3 How to Run

Step 1: Activate the conda environment.

```
conda activate <env-name>
```

Listing 3: Activate conda environment

Step 2: Navigate to the project directory.

```
cd path/to/project
```

Listing 4: Navigate to project

Step 3: Run the agent.

```
python test_agent.py
```

Listing 5: Run the agent

The script will run questions against both datasets and write results to `results.csv`.

2 Prompt Log

2.1 Prompt 1:

Goal: The initial implementation hardcoded `pd.read_csv('housing.csv')` in the generated code, ignoring the `csv_path` argument passed to `run_agent()`. The fix needed to make the generated code use the dynamically provided path so that the agent works correctly for both `housing.csv` and `custom_dataset.csv` without any structural changes.

Prompt:

```
The minimal implementation passed the first local test, but I found an important issue.

Current generated code is hardcoding:
pd.read_csv('housing.csv')

Instead, it must use the csv_path argument dynamically, because the assignment requires:
def run_agent(question: str, csv_path: str) -> dict

Please fix the implementation so that generated code uses the provided csv_path rather than hardcoding a filename.

Requirements:
- Keep the current structure.
- Do not rewrite the whole project.
- Make the smallest necessary changes.
- Preserve these assignment constraints:
  - run_agent(question: str, csv_path: str) -> dict
  - exec(code, env)
  - generated code must store output in result
- Also make sure this will work later for custom_dataset.csv.

Please show:
1. the root cause
2. the exact file changes
3. the updated code
4. how to rerun the same local test
```

Listing 6: Development prompt 1

Result: The response was helpful and worked directly without further modification. The generated code correctly used the injected `csv_path` variable, stored the final output in `result`, executed without errors via `exec(code, env)`, and returned all four required fields.

2.2 Prompt 2:

Goal: The answer generation step failed to correctly extract content from `execution_result` when it was a `DataFrame` with nested dict-like columns (`low_value_area`, `high_value_area`). The `final_answer` was producing empty sentences of the form *"The main differences appear in these representative areas: ; ; ."* The goal was to fix only the answer generation logic so that it could unpack nested dict values and produce a readable, semantically accurate comparison mentioning income bin, ocean proximity, median house value, and the gap.

Prompt:

```
The readability polish improved some outputs, but advanced test 3 still
has a clear summarization bug.

Current issue:
- execution_result is a DataFrame with nested dict columns such as
  low_value_area and high_value_area
- final_answer becomes:
  "The main differences appear in these representative areas: ; ; ."
- so the answer-generation logic is not correctly extracting content
  from nested dict values inside DataFrame rows

Please fix only the final_answer generation for this kind of result.

Requirements:
- Do not change generated_code, execution_result, evaluation,
  or the overall workflow
- Do not rewrite the whole project
- Make the smallest necessary change

What I want:
- If execution_result is a DataFrame with columns like low_value_area
  / high_value_area that contain dict-like objects, summarize the
  first 2-3 rows clearly in natural language
- mention:
  - the income bin
  - the low-value area's ocean_proximity and median_house_value
  - the high-value area's ocean_proximity and median_house_value
  - the approximate gap
- avoid dumping raw dict text

Example style:
"For income bin 7.0, an INLAND area with median house value around X
contrasts with a NEAR BAY area around Y, despite similar median income.
Similar gaps appear in ..."

Please show:
1. exact file changes only
```

```
2. a short explanation of the fix
3. how to rerun advanced test 3
```

Listing 7: Development prompt 2

Result: The response was partially helpful. The summarization bug was fixed and the output became readable. **However**, the semantic content regressed: the revised `final_answer` described generic characteristics (e.g., *"weaker incomes, older housing stock, denser conditions"*) without grounding claims in the actual data — no specific income bins, no concrete `ocean_proximity` values, no numeric house value figures from the top rows. Readability improved but analytical precision was lost.

2.3 Prompt 3

Goal: After the summarization fix in Prompt 2, the readability of `final_answer` improved, but the underlying analysis became semantically incorrect. The generated code was comparing a very low income band against a very high income band, which directly contradicts the question's requirement of *similar* median income levels. The goal was to fix only the analysis logic for Advanced Test 3 so that the comparison truly enforced a small income gap and a large house value gap simultaneously, without touching the overall workflow.

Prompt:

```
Advanced test 3 is now more readable, but the analysis no longer
matches the question.

Question: "Identify areas with similar median income levels but
significantly different median house value."

Current issue:
- the current code compares a very low income band and a very high
  income band
- these are not similar income levels
- so the result is easier to read, but semantically incorrect

Please fix only this advanced test 3 analysis so that it truly enforces:
- similar median income levels
- significantly different median house values

Requirements:
- Do not rewrite the whole project
- Do not change the overall workflow
- Make the smallest necessary change
- Keep the result structured and readable
- final_answer should remain natural-language, but execution_result
  must reflect the correct logic

```

```
Suggested direction:
- group areas by a narrower income band or rounded median_income
- compare areas within the same or very close income band
- only keep comparisons where income difference is small but house
  value gap is large

For the final result, I want:
- income band
- low-value area summary
- high-value area summary
- house value gap
- small income gap

Please show:
1. root cause of the semantic issue
2. exact file changes only
3. updated code
4. how to rerun only advanced test 3
```

Listing 8: Development prompt 3

Result: The response correctly identified the root cause — the previous fix had binned income too coarsely, placing groups with very different income levels into the same comparison.

Advanced Test 3 now returns structured comparisons that are semantically faithful to the question, and `final_answer` correctly summarizes the top representative pairs with income band, area characteristics, and value gap — with no empty separators.

2.4 Prompt 4:

Goal: The agent was tested on the custom dataset (Netflix) and failed. The generated code still referenced `df["ocean_proximity"]` and other housing-specific columns that do not exist in the custom CSV. The root cause was clear: the code generation prompt never instructed the model to inspect the actual schema of the provided CSV, so the model defaulted to the column names it had seen most during the fixed dataset tests. The goal was to make the smallest possible change so that the codegen node reads the current CSV schema first and injects column names, dtypes, and a row preview into the prompt before asking the model to generate code.

Prompt:

```
The custom dataset tests show that the current agent is overfitting
to the housing dataset.

Observed problems:
- For Netflix questions, generated code still uses
  df["ocean_proximity"], which does not exist.
- Another question returned None values instead of using the actual
  release_year column.
```

```
I need you to fix the code generation logic so it adapts to the
current CSV schema.

Requirements:
- Do not rewrite the whole project.
- Make the smallest necessary change.
- Keep the current overall workflow.
- The generated code must still be executed with:
    env = {}
    exec(code, env)
- The generated code must still store output in result.

What to improve:
1. Before generating code, inspect the current CSV schema.
2. Pass useful schema information into the code-generation prompt:
   - column names
   - dtypes
   - a small preview of rows
3. Update the prompt so the model is explicitly told:
   - use only columns that actually exist in the current dataset
   - do not assume housing.csv columns
   - adapt to the provided csv_path and schema
4. If a question refers to a concept not present in the dataset,
   handle that gracefully instead of hallucinating columns.

Please show:
1. the root cause
2. exact file changes only
3. the updated code
4. how to rerun the 3 Netflix basic tests
```

Listing 9: Development prompt 4

Result: The response was helpful and worked correctly after one application. The fix added a lightweight schema inspection step at the start of the codegen node: before constructing the LLM prompt, the node now reads the CSV with `pandas`, extracts column names, dtypes, and a two-row preview, and injects this information as a schema block into the code generation prompt. The model is explicitly instructed to use only columns present in the schema and not to assume any fixed dataset structure. After fixing, the three basic Netflix tests passed. Schema-aware code generation generalized cleanly to the custom dataset without any further prompt iteration.

2.5 Prompt 5:

Goal: Custom Advanced Test 2 asked for countries with few total titles but a high proportion of TV Shows. The generated code grouped by the raw `country` column directly, so entries like *"Ar-*

gentina, United States, Mexico" appeared as a single group key. This made the analysis semantically invalid — the question asks about individual countries, not co-production strings. The goal was to split multi-country cell values into individual entries before aggregating, with the smallest possible change to the existing code.

Prompt:

```
Custom advanced test 2 still has a semantic issue.

Question: "Find countries that have relatively few titles overall but
a high proportion of TV Shows compared with Movies."

Current issue:
- the result is grouping by the raw country cell string
- so outputs include values like "Argentina, United States, Mexico"
  instead of single countries
- that does not really answer the question about countries

Please fix only this analysis.

Requirements:
- split multi-country values into individual countries
- trim whitespace and remove empty strings
- compute total titles, movie_count, tv_show_count, and tv_share
  at the single-country level
- keep the idea of "relatively few titles overall" and
  "high proportion of TV Shows"
- keep the result structured and readable
- do not rewrite the whole project
- make the smallest necessary change

Please show:
1. the exact root cause
2. exact file changes only
3. updated code
4. how to rerun only custom advanced test 2
```

Listing 10: Development prompt 5

Result: The response was helpful and the fix worked on the first attempt. After the change, all group keys in the result were single country names. The aggregation of `total_titles`, `movie_count`, `tv_show_count`, and `tv_share` then operated correctly at the individual-country level, and Custom Advanced Test 2 passed with semantically valid output.

3 Failure Analysis and Debugging

3.1 Failure Case 1:

Question that failed:

"How does population density (defined as population per household) relate to the median value of the house?

Did the generated code run on the first attempt?

*Y*es, but the output was incorrect in design: the code stored a natural language explanation directly in `the result` (e.g., *"Population density has a weak negative correlation with the median house value"*), rather than a raw analytical value.

What went wrong:

First, the initial code generation prompt did not distinguish between raw analysis output and natural language interpretation, so the LLM sometimes collapsed both steps into one, storing an explanatory sentence in `result` instead of a numeric or structured value. Second, the answer generation step used `str(result)` directly, which worked correctly only when `result` was already a well-formed string.

How to debugg:

Before: The code generation prompt did not specify what type of value should be stored in `result`, leaving the choice to the LLM. The answer generation prompt applied `str(result)` directly without any interpretation step.

After: A targeted prompt was sent requesting the smallest necessary change:

```
The intermediate tests are passing, but I found a design issue.
For some questions, the generated code is directly producing
explanatory text as result, instead of returning raw analysis output.

Please make a small improvement:
- The generated code should prefer structured or raw results
  (number, dict, Series, DataFrame) stored in result.
- The final_answer step should be responsible for turning
  execution_result into natural language.
- Keep the current project structure.
- Do not rewrite the whole project.
- Make the smallest necessary change.
- Keep the implementation student-friendly.

Please show:
1. what the issue is
2. which file(s) need to change
3. the updated code
4. how to rerun the intermediate tests
```

Listing 11: Prompt used to fix the design issue

Did the revised prompt improve the result?

Yes. After the fix, the generated code stored a structured numeric value in `result`

The execution returned `np.float64(-0.04395404801584546)`, and the answer generation step correctly interpreted this as a weak negative correlation.

3.2 Failure Case 2:

Question that failed:

"Identify areas with similar median income levels but significantly different median house value. What factors might explain these differences?"

Did the generated code run on the first attempt?

No. The code raised a runtime error immediately upon execution.

What went wrong:

The generated code iterated over groups using `tmp.groupby("income_bin", observed=True)`, unpacking each group as `(_, g)` where `g` is a `DataFrame` slice. It then accessed `g.name` to retrieve the group key:

```
for _, g in tmp.groupby("income_bin", observed=True):
    ...
    "income_bin": str(g.name),
```

Listing 12: Buggy generated code

The root cause is that `g` is a `DataFrame`, and `DataFrame` objects do not have a `.name` attribute in this context — `.name` belongs to `Series`, not `DataFrame`. The group key is captured by the first unpacking variable (_), which was incorrectly discarded.

How to debugg:

The traceback pointed directly to `str(g.name)` inside the loop body. Inspecting the `groupby` unpacking convention confirmed that the first element of each tuple is the group key (a scalar), and the second is the `DataFrame` slice. The fix required only renaming the discarded variable and substituting it in place of `g.name`.

```
The advanced test 3 failed with this error:
    AttributeError: 'DataFrame' object has no attribute 'name'

The generated code includes:
    for _, g in tmp.groupby("income_bin", observed=True):
        ...
        "income_bin": str(g.name),

Please debug this carefully.

Requirements:
```

```
- Do not rewrite the whole project.
- Identify the exact root cause.
- Make the smallest possible fix.
- Preserve the current workflow and structure.

Also, after fixing the execution bug, improve the result so that it
better matches the question:
"Identify areas with similar median income levels but significantly
different median house value. What factors might explain these
differences?"

That means:
- prefer grouped/area-level analysis over raw rows
- keep the result structured
- let final_answer do the explanation

Please show:
1. root cause
2. exact file changes
3. updated code
4. how to rerun the test
```

Listing 13: Prompt used to fix and improve the advanced question

Did the revised prompt improve the result?

Yes. Beyond the bug fix, the revised result became substantially more meaningful: the output now contains two structured components — `comparisons`, which pairs low-price and high-price areas within the same income bin to directly address the "similar income, different value" framing, and `factors`, which aggregates median contextual features by `ocean_proximity` to support the explanatory part of the question. The answer generation step then interprets this structure into a coherent natural language response.

4 Self-Grading

4.1 Fixed Dataset (housing.csv)

#	Question	Pass/Fail	Notes
1	Simple Q1	PASS	
2	Simple Q2		
3	Simple Q3	PASS	
4	Intermediate Q1	PASS	
5	Intermediate Q2	PASS	
6	Intermediate Q3	PASS	
7	Advanced Q1	PASS	

#	Question	Pass/Fail	Notes
8	Advanced Q2	PASS	
9	Advanced Q3	PASS	

4.2 Custom Dataset (custom_dataset.csv)

The custom dataset used in this project is the **Netflix Titles** dataset, a publicly available collection of movies and TV shows available on Netflix. The dataset contains 8,807 rows and 12 columns, including: `show_id`, `type`, `title`, `director`, `cast`, `country`, `date_added`, `release_year`, `rating`, `duration`, `listed_in`, and `description`.

#	Question	Pass/Fail	Notes
1	Simple Q1	PASS	
2	Simple Q2	PASS	
3	Simple Q3	PASS	
4	Intermediate Q1	PASS	
5	Intermediate Q2	PASS	
6	Intermediate Q3	PASS	
7	Advanced Q1	PASS	
8	Advanced Q2	PASS	The sample size is small, and the explanatory power is weak.
9	Advanced Q3	PASS	

5 Advanced Question Reflection

5.1 Fixed Dataset

5.1.1 Advanced Q1

The answer is not the only possible correct answer because the phrase “geographic areas” is not uniquely defined in the question.

The question can be interpreted in multiple ways. “Geographic areas” could mean latitude/longitude grid cells, named regions, coastal categories, counties, or clusters derived from the data

Different answers could be valid if they use a reasonable area definition and explain the ranking using available variables such as median income, housing age, rooms, population, households, and ocean proximity. For example, using 0.5-degree bins versus 10 equal-width bins would likely change the exact top five areas, but both approaches could still be justified.

A less ambiguous version would be: “Divide the dataset into latitude-longitude grid cells using 0.5-degree bins. Identify the top 5 grid cells with the highest median house value, and summarize how median income, housing median age, total rooms, total bedrooms, population, households, and ocean proximity differ in those cells.”

5.1.2 Advanced Q2

The answer is not unique because both “coastal areas” and “relatively low” depend on modeling choices.

Different answers could be valid if they clearly define coastal groups and use a reasonable threshold for low prices.

A clearer version would be: “Using only rows with ocean proximity categories NEAR OCEAN, <1H OCEAN, or ISLAND, group the data into latitude-longitude bins and identify the 10 coastal groups with the lowest median house value.

5.1.3 Advanced Q3

The answer depends on how I define “similar median income levels” and “significantly different median house value.” In my final version, I used rounded income bins and kept comparisons where the income gap was small and the house value gap was large.

The question can be interpreted as comparing geographic bins, comparing coastal categories, or comparing individual observations with close income values.

Different answers could be valid if they enforce a principled similarity rule for income and a principled threshold for house value difference.

A more precise version would be: “Round median income to the nearest 0.1 and compare areas only within the same rounded income bin.

5.2 Custom Dataset

5.2.1 Advanced Q1

It correctly identified the top 5 release years; the results are clear and align with the problem statement.

The top 5 release years by count are objectively determined by the dataset, so that ranking is relatively fixed. However, the explanation of what those peaks “suggest” is not uniquely determined by the data.

Different interpretations could be valid. One explanation might emphasize Netflix’s focus on more recent content, while another might emphasize catalog expansion, acquisition of licensed back-catalog titles, or increased content diversity.

A clearer version would be: “Identify the top 5 release years with the highest number of titles in the dataset. Then provide a brief hypothesis, based only on the observed counts, about what this pattern may indicate about the composition of Netflix’s catalog.”

5.2.2 Advanced Q2

As a result, the explanatory scope is limited due to the influence of the small sample size. And the answer is not unique.

The question can be interpreted differently depending on whether titles with multiple countries are split into separate countries or treated as combined country strings.

Different answers could be valid if they apply a clear and reasonable rule.

5.2.3 Advanced Q3

It reveals the years in which the distribution of movies and TV shows differs significantly.

The answer depends on how “very differently” is measured. In my implementation, I used share gaps between Movies and TV Shows and filtered out years with very small total counts. A different threshold or ranking criterion could produce a different set of years.

Different answers could be valid if they use a clearly defined measure of imbalance.

How to rewrite:“For release years with at least 50 total titles, calculate the share of Movies and TV Shows. Identify the 10 release years with the largest absolute difference between these shares, and briefly explain what this pattern may indicate about shifts in the composition of the catalog.”